

FlagAI powered by The Call AI

A game-day referee app: rules, rosters, and game timing • SIP311 Round 2 re-pitch



Joseph Hudson Jr.

University of Advancing Technology

B.S. Artificial Intelligence & B.S. Advancing Computer Science, SIP311



Round 2: What Changed

Round 1 feedback was that rules alone can be looked up with existing AI tools, and that rulebook material carries IP risk. I agree. FlagAI is now a referee's game-day app, with rosters and game timing built in alongside the rule engine, and it is built in Python with Flask.

How each condition is addressed

- **Referee app with rosters and timing:** the rule engine runs inside a live game situation. A timing module runs the game clock, play clock, timeouts, and two-minute warning; a roster module checks numbering, eligibility, and disqualification counts; a foul log keeps the record.
- **Rulebook and IP:** rules are encoded as my own Python logic keyed by rule-section-article and shown with summaries in my own words. No rulebook text is stored or displayed and no league marks are used. The knowledge base is ruleset-agnostic, so an NFHS high school ruleset can be added without changing the engine.
- **Python + Flask:** the whole MVP is Python. Flask serves the web app with Jinja templates, and the same process calls the Claude API for parsing, which removes the separate React front end.
- **This pitch:** five minutes on features, the innovation claim, and the programming languages.

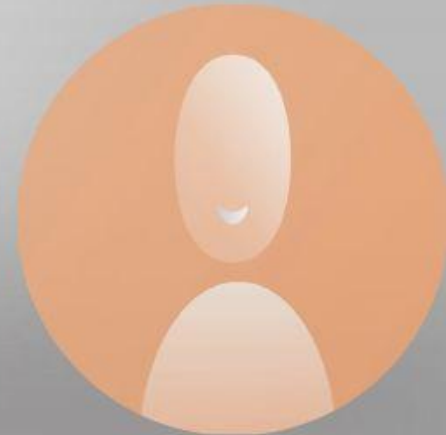


Features: The Game-Day App

FlagAI is what an official opens on a phone or tablet at the game. It keeps the clock, knows who is on the field, rules on the play, computes the enforcement, and keeps the record.

MVP features

- **Game timing:** game clock with the Rule 4 stop and start reasons, the 40- and 25-second play clock, three timeouts per half, two-minute warning, and period changes. Every ruling reports what the clock does next.
- **Rosters:** both teams' numbers and positions. Checks eligibility by numeral (Rule 5-1-2), counts players on the field (5-1-1), tracks reported position changes (5-3-1), and warns when a second unsportsmanlike foul would disqualify a player (12-4-1).
- **Make the Call:** type the play in plain English; FlagAI identifies the foul and computes the enforcement against the live situation: spot, yardage, resulting down and distance, first-down status, and clock, in under five seconds.
- **Rule reference and explanation:** each ruling cites the rule by number with a plain-language summary, explains why, and states what facts would change the call.
- **Foul log and post-game report:** every accepted ruling is logged with time, team, and player number, exportable for the crew.

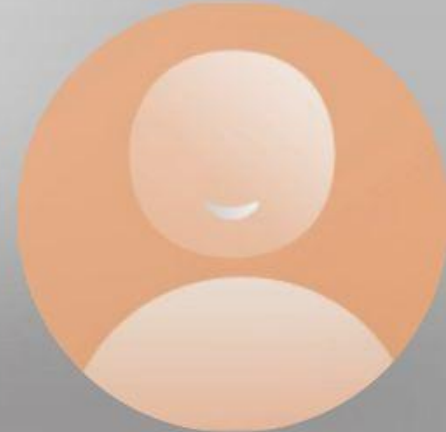


Innovation Claim

FlagAI, powered by The Call AI, is the first referee app whose rulings come from a deterministic, game-aware rule engine rather than a chatbot. A general AI tool can answer a rules question; it cannot take the live clock, down, distance, spot, and roster, apply the enforcement, and hand the official the new game state with an audit trail.

What makes it innovative

- **Game-aware, not lookup:** the ruling is computed against the actual situation and roster, so the output is the enforced result (1st and 10 at the 15, clock starts on the snap, player at one foul from disqualification), not a paragraph about the rule.
- **Deterministic and testable:** the decision is Python rule code, validated against a 200-play corpus in CI. The same facts always produce the same ruling; the LLM only parses the play and explains the result.
- **One tool for the whole game:** timing, rosters, and rulings share one game-state object, so the app replaces the stopwatch, the roster card, and the rulebook search at once, in a browser with no cameras or sensors.
- **Explainable and safe on IP:** every ruling shows the rule number, a summary in FlagAI's own words, and what would change the call, without reproducing rulebook text.



Technical Approach: Python + Flask

One language end to end. Python handles the rule engine, the timing and roster logic, the LLM calls, the web app, and the tests, which is the right shape for a solo build and the simplest path to integrating the AI model.

Languages and frameworks

- **Python 3.12:** rule engine, timing, and roster modules as pure functions over a typed game-state object; Claude API client for play parsing with JSON-schema validation; advisory layer.
- **Flask + Jinja2:** routes, forms, and server-rendered templates for the game-day screen; HTML and a small amount of JavaScript for the running clocks.
- **SQL:** SQLite for the MVP, PostgreSQL for deployment; SQLAlchemy models for games, rosters, situations, rulings, and the foul log.
- **Testing and delivery:** pytest against the labeled play corpus and timing scenarios, GitHub Actions on every commit, Docker image deployed to Render, secrets in environment variables.

Why Flask over FastAPI + React

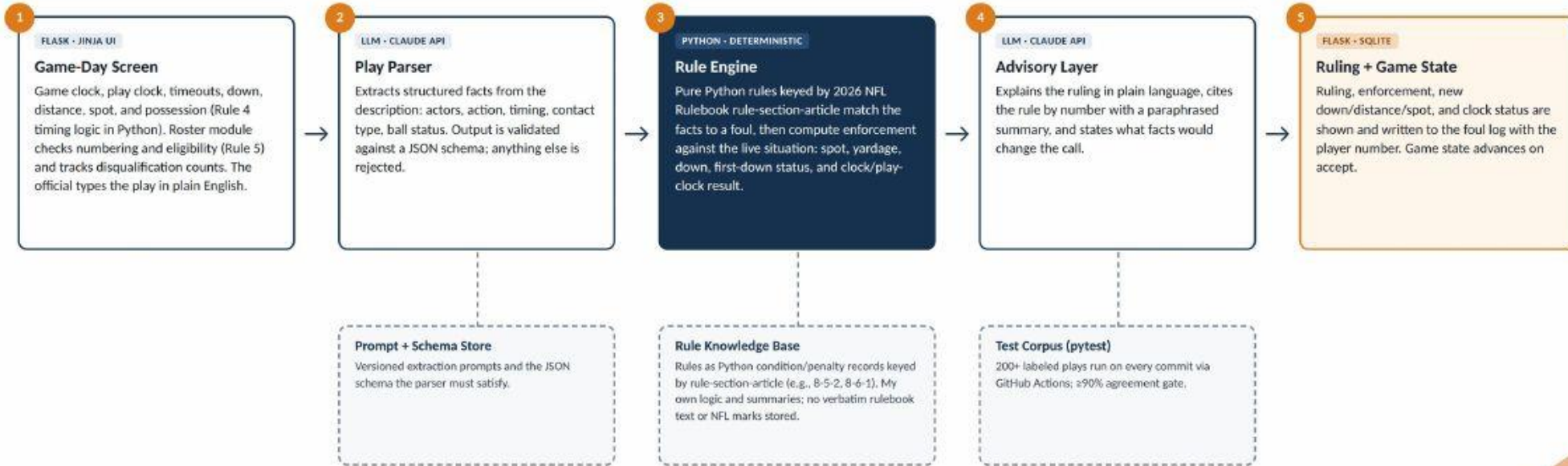
- **One process:** the parser, the engine, and the page live in one Python process, so there is no API contract to maintain between a JavaScript front end and the model. Fewer moving parts, faster iteration, and everything I write during SIP is Python.



Proof of Concept: MVP Architecture

FlagAI powered by The Call AI — MVP Architecture (Python + Flask)

Game timing + rosters + play description in → deterministic ruling, enforcement, and updated game state out. The LLM never decides the call; it parses and explains.



Stack: Python 3.12 · Flask · Jinja2 · SQLite → PostgreSQL · Claude API · pytest · GitHub Actions · Render

■ deterministic core □ LLM-assisted ■ user-facing

System flow for the MVP in Python and Flask. Timing and roster modules feed one game-state object; the LLM parses the play and explains the result; the ruling and enforcement come from deterministic Python rule code keyed by rule-section-article, validated against a 200-play test corpus.

Early-Stage Prototype: Game-Day Screen

FlagAI

powered by The Call AI · Game-day referee app · v0.2 prototype

Home vs Visitor · Ruleset: 2026 NFL (rule-section-article) · Crew: R

GAME SITUATION & TIMING

Down

3rd

Distance

7

Ball on

V-42

Possession

Home

07:42 · Q2

Clock stopped: incomplete pass (4-4-f) · Line to gain V-35
Timeouts H 3 · V 2 (4-5-1) · Two-minute warning pending (3-41)

:25

PLAY CLOCK · 4-6-2

DESCRIBE THE PLAY

QB throws deep down the left sideline. At the V-15, before the ball arrives, #21 grabs the receiver's arm and turns him. Pass falls incomplete.

MAKE THE CALL

ROSTER CHECK · VISITOR

#21 CB · DB 0-49

eligible · UNS 0/2

#77 OT · OL 50-79

ineligible (5-1-2)

#91 DE · 90-99

UNS 1/2 · next = DQ (12-4-1)

On field

11 of 11 (5-1-1) ✓

FOUL LOG · THIS GAME

Q2 09:15 · Holding (O) #64 H

10 yds · 2nd & 18

Q1 03:02 · False start #77 H

5 yds · 1st & 15

Q1 08:40 · Taunting #91 V

15 yds · UNS 1

Q1 11:10 · Offside (D) #91 V

5 yds · 1st & 5

FLAG · DEFENSE · #21 V

SPOT FOUL

Defensive Pass Interference

Defender grabbed the receiver's arm and turned him while the pass was in the air, more than one yard beyond the line, restricting his opportunity to catch the ball.

ENFORCEMENT

Spot of the foul

ball placed at V-15 (8-6-1-b)

RESULT

1st & 10 at V-15

automatic first down for the offense

CLOCK

Starts on snap

25-sec play clock · parser confidence 94%

RULE REFERENCE · 8-5-1 (DEFINITION) · 8-5-2 (c), (f) · PENALTY 8-5 · SPOT 8-6-1 (b)

In FlagAI's words: once a forward pass is thrown, a defender more than one yard past the line may not grab a receiver's arm or hook him so that his body turns before the ball arrives. The offense gets a first down at the spot of the foul. This is FlagAI's summary; the official text is in the 2026 NFL Rulebook.

WHY THIS CALL · WHAT WOULD CHANGE IT

Contact was defender-initiated, after the throw, beyond one yard, and restricted the receiver. **No foul** if the pass was clearly uncatchable or the contact was incidental with both playing the ball (8-5-3 a, c). **Defensive holding** (5 yards, automatic first down; 12-1-6) if the grab happened before the pass was thrown.

ACCEPT · ADVANCE GAME STATE

OVERRIDE

Flask prototype mockup: game clock, play clock, roster check, and play description in; ruling, rule reference by number, enforcement computed against the situation, foul log, and accept/override out. SIP311 build target: the rule engine with 10 fouls, the timing module, and the Flask game-day screen.